IT Helpdesk Software Specification

Employee issue reporting and resolution tracking

Version 1.0 | 29 October 2026 | Draft for stakeholder review

Purpose and intended outcome

Build a web application that allows employees to report IT issues, track their ticket status, and receive updates. IT support staff use the same application to review, assign, progress, and resolve tickets. This document defines observable behavior and verification criteria for an initial release, suitable for planning, AI-assisted implementation, and testing.

Source requirement

“Employees need a way to report IT issues, track status and get updates.”

Source: the user-provided slide titled What Is Specification-Driven Development (SDD). The slide also identifies ticket creation, viewing and updates; required-field validation; permissions; a web form and REST API; a new-ticket status of Open; and valid, invalid and unauthorized test cases.

How to read this specification

Requirements marked S come directly from the slide. Requirements marked P are proposed elaborations needed to make the brief implementable and testable. P requirements are a coherent draft baseline, not stakeholder-approved facts. All field lengths, roles, notification behavior, performance targets, and operational policies are proposed unless explicitly identified as S.

“Shall” describes required behavior within this draft baseline. Examples illustrate the contract; they do not add features. Requirement IDs connect implementation work, acceptance criteria, and tests. The product need does not require an AI chatbot or AI-powered ticket resolution; AI-assisted development is the delivery approach shown in the slide.

Document map

Section	Coverage
1–2	Scope, assumptions, actors and access
3–5	Functional requirements, workflow and screens
6–8	Data model, REST interfaces and reliability
9–10	Quality requirements, acceptance criteria and test cases
11	Delivery sequence, change control and decisions

1 Scope and baseline assumptions

Included in the initial release

Employees create and view their own tickets, add comments, and read updates. Support agents view all tickets in the single organization, assign work, update priority and status, and record resolution details. The application provides a persistent ticket history and in-app notifications. All user-facing actions are enforced by server-side permissions.

ID	Basis	Requirement
S01	S	Employees shall be able to report IT issues.
S02	S	Employees shall be able to track issue status.
S03	S	Employees shall be able to receive updates.
S04	S	The system shall support creating, viewing and updating tickets.
S05	S	Required fields shall be validated and permissions enforced.
S06	S	The system shall expose a web form and REST API.
S07	S	Every new ticket shall start with status Open.
S08	S	Verification shall include valid, invalid and unauthorized actions.

Proposed assumptions

A01 — This release serves one organization. Each account has exactly one application role: Employee or Support Agent. Identity and roles are provisioned outside the application; there is no user-management screen.

A02 — The three visible statuses are Open, In Progress and Resolved. Priority is Low, Medium or High, with Medium selected initially. These values are derived from the slide examples and adopted here as the proposed complete sets.

A03 — “Get updates” means persistent in-app notifications and ticket history. Email, SMS, WhatsApp and push notifications are excluded until explicitly requested.

A04 — Support agents may work on any ticket. Assignment indicates responsibility and does not grant exclusive access. Users can also raise their own tickets regardless of role.

Excluded from this release

File attachments, anonymous tickets, internal-only comments, deletion, automatic assignment, escalation rules, SLA timers, approvals, knowledge articles, email ingestion, integrations, multi-organization tenancy, and AI triage are excluded. Reports appears in the illustrative slide navigation but has no defined business requirement; it is excluded and shall not appear as a nonfunctional menu item.

2 Actors and access control

P01 — Every page and API operation containing ticket or notification data shall require an authenticated, active account. The server derives the user ID and role from validated identity context. Client-supplied identity or role values shall never determine access.

Capability	Employee	Support Agent
Create a ticket	Yes, for self	Yes, for self
List and read tickets	Own tickets only	All tickets
Add public comment	Own tickets only	Any ticket
Change title or description	No after submission	No after submission
Change priority	No after submission	Any ticket
Assign or reassign	No	Any active Support Agent
Change ticket status	No	Any ticket within transition rules
Read history	Own tickets only	All tickets
Read or mark notifications	Own notifications only	Own notifications only
Delete ticket or history	No	No

Authorization rules

P02 — Ownership is the immutable requester_id stored at creation. An Employee shall see only tickets they created, including through search, pagination totals, history and comments. A Support Agent shall see all tickets. UI visibility is a convenience; the API must independently enforce the same rules.

P03 — Unauthenticated or expired sessions shall receive HTTP 401. Authenticated users attempting a role-restricted endpoint shall receive 403. Requests for a nonexistent ticket or a ticket the user cannot read shall return the same 404 response. Authorization is checked before mutation.

P04 — Notification ownership is always personal. Support Agent status does not allow reading another user’s notifications. Inactive users cannot authenticate or be assigned new work; existing historical actor references remain visible.

Primary user journeys

Employee: sign in → open New Ticket → enter issue details → submit → receive reference and Open status → find ticket in My Tickets → read progress and comments → see resolution and provide follow-up comments if needed.

Support Agent: sign in → open All Tickets → review unassigned tickets → assign an agent → set In Progress → add updates → record resolution → set Resolved. If the issue recurs, an agent reopens the ticket after reviewing the employee’s comment.

3 Functional requirements

Create and retrieve tickets

FR01 [S01, S04, S07] — On valid submission, the server shall create one ticket with a unique immutable UUID, a readable reference, the authenticated requester, status Open, an initially null assignee, and server-generated UTC timestamps. A success response shall contain the saved ticket; the form shall navigate to its details.

FR02 [S05] — Title, description and priority shall pass the field rules in Section 6. Missing, blank, out-of-range or unknown fields shall fail validation without creating a ticket. No partial ticket, history entry or notification shall remain after a failed transaction.

FR03 [S02, S04] — My Tickets shall display reference, title, status, priority and last updated time. It shall support status and priority filters plus reference/title search. Detail shall display the original description, requester, assignment, resolution information and chronological activity.

FR04 [P] — All Tickets shall be available only to Support Agents. In addition to shared filters, it shall support assigned-to-me and unassigned filters. Filtering and pagination shall never reveal inaccessible records.

Manage support work

FR05 [S04; P rules] — Support Agents shall be able to set assignee, priority and status using a partial ticket update. At least one supported field shall be supplied. Only actual changes shall generate update events. Every modification shall use the ticket version to prevent overwriting a newer change.

FR06 [P] — The assignee shall be an active Support Agent or null where the lifecycle permits. Reassignment shall record the previous and new assignee. The original requester remains unchanged.

FR07 [S02; P rules] — Status changes shall follow Section 4. Resolution requires a meaningful resolution summary. Invalid transitions shall not change the ticket. Previously recorded resolution events remain available after reopening.

Conversation and updates

FR08 [P] — An authorized user shall be able to append a plain-text public comment to a ticket in any status. Comments are immutable, visible to everyone authorized to read that ticket, and ordered by creation time then ID. Adding a comment to a Resolved ticket shall not automatically reopen it.

FR09 [S03; P channel] — Ticket changes shall create persistent in-app notifications according to Section 8. Users shall be able to view their notifications, distinguish unread items, and mark their own items as read.

FR10 [P] — Creation, field changes, status transitions and comments shall produce immutable activity events with actor and server time. Assignment and priority events show old and new values; status events show old and new status and any resolution summary.

4 Ticket lifecycle and business rules

From	To	Conditions
New submission	Open	Server-controlled; never supplied by client.
Open	In Progress	Active Support Agent assignee required.
In Progress	Resolved	Assignee retained; resolution summary required.
Resolved	Open	Reopen reason required; current resolution cleared; assignee retained if still active, otherwise cleared.

P05 — All other status transitions are rejected with 409 INVALID_TRANSITION. In particular, Open cannot move directly to Resolved. An update that supplies the existing status is allowed as a no-op for that field and does not require a new transition reason.

P06 — A Support Agent may assign a ticket and move it from Open to In Progress in one request. Validation applies to the final proposed state, so the result must have an active Support Agent assignee. In Progress tickets cannot be unassigned, but can be reassigned.

P07 — Resolved tickets retain their last resolution summary and resolved_at timestamp. Their assignee, priority and status are read-only except for the permitted reopen operation. Comments remain allowed. Reopening clears the current resolution_summary and resolved_at fields, while history preserves the previous resolution.

P08 — Resolution summaries and reopen reasons shall contain 10–2,000 characters after trimming. resolution_summary is accepted only when entering Resolved. reopen_reason is accepted only when entering Open from Resolved. Supplying these fields in other situations returns 422.

Atomic updates

P09 — The ticket mutation, version increment, activity event and corresponding notification records shall commit atomically. If any required write fails, all writes shall roll back. A successful response must reflect the committed state.

P10 — Every successful ticket field change or comment increments version by one and updates updated_at. A patch containing only unchanged values returns the current ticket without incrementing its version, adding activity or sending notifications. Marking a notification read never changes the ticket.

Example scenario

An employee submits “VPN disconnects during calls” with Medium priority. The ticket starts Open. A support agent assigns it and moves it to In Progress. The agent adds a comment requesting a reconnect attempt. After confirming the fix, the agent submits a resolution summary and changes the status to Resolved. If the employee reports a recurrence, an agent reopens the ticket with a reason; the previous resolution remains in history.

Handling repeat submissions

P11 — The UI disables Submit while a request is pending. The API additionally supports an Idempotency-Key for ticket creation and comments. Retrying the same request with the same key shall return the original result without creating duplicate records or notifications. Detailed rules appear in Section 8.

5 Web interface specification

Screen	Required content and behavior
Sign in	Use configured identity provider. Return to the intended authorized page after authentication.
New Ticket	Required Title, Description and Priority controls; Medium initially selected; Create Ticket action; field-level errors.
My Tickets	Own tickets, filters, search, pagination and New Ticket action. Default sort is most recently updated first.
All Tickets	Support Agent only. All authorized tickets with assignment filters and access to ticket details.
Ticket Details	Reference, title, description, status, priority, requester, assignee, timestamps, resolution and activity. Comment form for authorized users; update controls for agents.
Notifications	Newest first, unread indicator and mark-as-read action. Each item links to the ticket; unavailable targets show a generic unavailable message.

Form behavior

UI01 — Required fields shall be visibly labelled. Whitespace-only input shall fail validation. Client-side validation shall mirror server-side rules, but the server remains authoritative. Text entries shall be retained when validation or network errors occur.

UI02 — A successful creation shall show the ticket reference and Open status. The page shall never imply success until the server confirms the write. On timeout, retain the original idempotency key so Retry can safely recover the result.

UI03 — On a version conflict, display “This ticket has changed. Refresh it before saving.” Preserve unsent comment or update input separately, load current data, and require the user to submit a revised action. Never silently overwrite another change.

Empty loading and error states

UI04 — Empty ticket lists shall say “No tickets found” and allow users to clear filters. New accounts with no tickets shall see a Create Ticket action. Loading indicators shall appear while data is loading. Errors shall offer a retry action without exposing stack traces or database details.

UI05 — A 401 response shall return the user to sign in. A 403 shall explain that the action is unavailable. A 404 shall show “Ticket not found or unavailable.” Failed mutations shall leave displayed persisted values unchanged until a fresh response is available.

Navigation and usability

UI06 — Employee navigation shall include New Ticket, My Tickets and Notifications. Support Agents additionally see All Tickets. All forms and controls shall support keyboard use, visible focus, associated labels and readable validation messages. Status must be expressed as text as well as color.

UI07 — Dates are stored in UTC and displayed in the user’s browser timezone with an accessible timezone indicator. Lists and detail views shall refresh automatically every 30 seconds while active, and support manual refresh. Background tabs may pause polling and refresh when activated.

6 Data model and validation

All identifiers below are opaque UUIDs unless stated otherwise. Text length means Unicode code points after trimming leading and trailing whitespace. Text is stored as plain text and escaped on display. Timestamps use ISO 8601 UTC values.

Ticket field	Type and rule
id / reference	Server UUID / unique reference HD- plus at least six decimal digits; sequence may contain gaps and shall never be reused.
title	Required string, 5–150 characters; immutable after creation.
description	Required string, 10–5,000 characters; immutable after creation.
priority	Required enum Low, Medium, High. UI defaults to Medium; API omission is invalid.
status	Server defaults to Open; enum Open, In Progress, Resolved.
requester_id	Required user foreign key from authentication; immutable.
assignee_id	Nullable user foreign key; active Support Agent on assignment.
created_at / updated_at	Server timestamps. created_at immutable; updated_at changes on ticket mutation or comment.
resolved_at	Nullable; set when resolved, cleared when reopened.
resolution_summary	Nullable text; required only for current Resolved status, 10–2,000 characters.
version	Positive integer; starts at 1 and increments on committed changes.

Related records

User: id, external_subject (unique identity-provider identifier), display_name, role and active. User provisioning is outside scope. Foreign keys preserve historical identity references; deactivation does not delete ticket history.

Comment: id, ticket_id, author_id, body (1–2,000 characters), created_at. Comments cannot be edited or deleted in this release.

ActivityEvent: id, ticket_id, actor_id, event_type, occurred_at, changed_fields containing old/new values, and optional comment_id or reason. Events are append-only. A patch may produce one event containing multiple changes; a comment produces one event linked to the comment.

Notification: id, recipient_id, ticket_id, event_id, summary, created_at and nullable read_at. Enforce uniqueness on recipient_id plus event_id. Summary shall omit description and comment text.

IdempotencyRecord: user_id, operation, key, request_hash, response_status, response_body and expires_at. Enforce uniqueness on user_id plus operation plus key for its retention window.

Persistence constraints

Enforce unique ticket IDs and references, valid enum values, required fields and foreign keys at persistence boundaries. Index requester_id, assignee_id, status and updated_at to support scoped lists. Activity and comments shall not be orphaned; no ticket deletion endpoint is included.

7 REST API contract

Base path: /api/v1. Requests and responses use application/json. All endpoints require authentication. Ticket responses expose fields from Section 6, with display names supplied alongside user IDs. Lists return {items, page, page_size, total}. Default page=1 and page_size=20; maximum page_size=100. Invalid parameters return 422.

Method and path	Contract
POST /tickets	Create for current user. Body: title, description, priority. Idempotency-Key required. Returns 201, ticket body and Location header.
GET /tickets	Default scope=mine. scope=all is agent-only. Optional status, priority, q, assignee=me|unassigned and pagination; assignee filter requires scope=all. Returns 200.
GET /tickets/{id}	Return accessible ticket or 404. Includes ETag holding its quoted version.
PATCH /tickets/{id}	Agent-only. Body subset: priority, assignee_id, status, resolution_summary, reopen_reason. If-Match required. Returns 200 with saved ticket and ETag.
GET /tickets/{id}/activity	Return activity with embedded comment body when relevant; chronological ascending by occurred_at then id; paginated.
POST /tickets/{id}/comments	Body: body. Idempotency-Key required. Returns 201 comment and new ticket_version. Any authorized reader may comment.
GET /notifications	Current user only. Optional unread=true|false and pagination. Returns 200; newest created_at first, then id.
PATCH /notifications/{id}	Body: {"read":true}. Sets read_at once; repeats return unchanged item. Returns 200, or 404 if unavailable.
GET /support-agents	Agent-only. Returns active agent IDs and display names, ordered by name then ID; paginated for assignment control.

Filtering and ordering

API01 — q is a literal, case-insensitive substring match on title or reference, limited to 100 trimmed characters. Empty q means no text filter. Multiple filters combine with AND. Ticket lists order by updated_at descending then id descending. total counts only matching authorized records. Pagination uses the current state; records may move between pages after concurrent updates.

Request examples

Create: {"title":"VPN disconnects during calls", "description":"The VPN disconnects every few minutes during video calls.", "priority":"Medium"}

Resolve: PATCH /tickets/{id} with If-Match: "3" and {"status":"Resolved", "resolution_summary":"Updated the VPN client and verified a stable connection."}. Requires the ticket to be In Progress and version 3.

8 Errors notifications and reliability

Error contract

All errors use {"error":{"code":"...","message":"...","fields":{},"request_id":"..."}}. fields maps invalid field names to plain-English messages; it is empty for nonvalidation errors. Unknown JSON fields are rejected, including requester_id or status in a create request.

HTTP	Code and meaning
400	MALFORMED_JSON — request cannot be parsed.
401 / 403	UNAUTHENTICATED / FORBIDDEN — missing identity or disallowed role/action.
404	NOT_FOUND — nonexistent or inaccessible resource.
409	INVALID_TRANSITION or IDEMPOTENCY_CONFLICT — business transition invalid or key reused with a different request.
412 / 428	VERSION_CONFLICT / PRECONDITION_REQUIRED — stale If-Match or required If-Match missing.
422	VALIDATION_ERROR — invalid fields, filters, enum, assignee or missing idempotency key.
500 / 503	INTERNAL_ERROR / UNAVAILABLE — unexpected failure or temporary dependency outage; no internal details.

Concurrent updates and retries

API02 — If-Match shall contain the quoted integer version returned by ticket reads. Version comparison and update happen atomically. A stale version returns 412 with no mutation. A malformed version returns 422. Authorization is checked before version details are exposed.

API03 — Idempotency-Key is a client-generated UUID, scoped by user and operation (including ticket ID for comments), retained for 24 hours. An exact replay returns the original status and body; a different body returns 409. Simultaneous requests with the same key are serialized. Replays recheck current authentication and resource authorization. Validation failures do not consume keys; after expiry, a new submission can create a new record.

Notification rules

Committed event	Recipients
Ticket created	Requester receives a submission confirmation.
Status or priority changed	Requester and current assignee, excluding actor.
Assignment changed	Requester, previous assignee and new assignee, excluding actor and inactive users.
Comment added	Requester and current assignee, excluding author.

N01 — For a patch with several changes, use the union of applicable recipients and create at most one notification per recipient per event. Null recipients are ignored. A no-op patch sends none. Notifications shall be available through the API immediately after commit and appear on an active UI within the 30-second refresh interval under normal service conditions.

N02 — Notification summaries include the ticket reference, event type and new status where applicable. Mark-as-read is explicit; merely viewing the list does not mark items read. Notification delivery failure shall roll back the associated mutation under this initial single-database design.

9 Quality and operational requirements

The following are proposed release targets. They are measurable acceptance gates, not claims about an existing system. Infrastructure sizing and production ownership must be settled before deployment.

ID	Requirement and verification
NF01 Security	Use HTTPS outside local development. Validate identity, authorization and input on every request; use parameterized database access. Escape rendered text. Test script-like payloads remain inert.
NF02 Sessions	Use an organizational identity provider; no local password storage. If cookie authentication is chosen, use Secure, HttpOnly, appropriate SameSite settings and CSRF protection for writes. Restrict allowed web origins.
NF03 Performance	In a stated test environment with 10,000 tickets, 50,000 activity records and 50 concurrent users, p95 read responses ≤1 second and writes ≤2 seconds over 15 minutes; unexpected failures <1%. Exclude external sign-in latency.
NF04 Usability	At viewport widths 375 and 1440 pixels, all primary actions remain usable without page-level horizontal scrolling. Test keyboard-only creation, filtering, commenting and agent updates.
NF05 Durability	A successful write survives application restart. Automated daily backups; proposed recovery point ≤24 hours and recovery time ≤4 hours. Verify a restore before production launch.
NF06 Observability	Log request ID, route, status, duration and operational error category. Do not log tokens, descriptions, comment bodies or resolution text. Monitor API failure rate and database availability.
NF07 Consistency	Ticket, activity, notification and idempotency success record commit in a single transaction. Simulated write failure shall leave no partial result.
NF08 Verification	Automate API validation, permission, lifecycle, retry and concurrency tests. Run UI smoke tests for Employee and Support Agent flows before release.

Architecture boundaries

A web frontend calls the REST API. The API owns authorization, validation, workflow and transactions. A relational datastore persists tickets and related records. An external identity provider authenticates users. These are logical responsibilities; the specific language, frontend library, database engine and hosting platform remain implementation decisions.

The Python and FastAPI code shown in the slide is illustrative and does not establish a mandatory technology stack. Implementation planning shall select a stack without weakening this contract. No external email service is required for the proposed initial release.

Deployment and data handling

Maintain separate development, test and production configuration; keep secrets outside source control. Apply versioned database migrations with a tested backup and rollback procedure. The runtime account shall have only necessary database permissions. Define retention and administrative data-removal procedures before production; the absence of an in-app delete button does not establish indefinite retention.

10 Acceptance criteria and test cases

Tests use separate Employee A, Employee B and Support Agent accounts, plus an inactive account. Seed distinct tickets owned by each employee. Each test below is mandatory for this proposed baseline; verify persisted state as well as the HTTP response and visible UI behavior.

Test	Given and action	Expected result and trace
TC01	Employee submits valid title, description and priority.	201; unique reference; Open; requester from session; event and confirmation. FR01, S07.
TC02	Submit blank title, short description, missing priority or unknown field.	422 with field error; zero new records. FR02.
TC03	Test title lengths 4, 5, 150, 151 and description 9, 10, 5000, 5001.	Only inclusive allowed boundaries pass. FR02, Section 6.
TC04	Submit without authentication; then as inactive user.	401; no ticket or related records. P01.
TC05	Employee A requests B’s ticket, activity or comment creation.	404 for each; no data exposure or mutation. P02–P03.
TC06	Employee requests scope=all or attempts ticket patch.	403; state unchanged. P02, FR04–FR05.
TC07	Employee lists tickets with search, filters and pagination.	Only own matches; correct total, ordering and empty state. FR03, API01.
TC08	Agent assigns active agent and moves Open to In Progress atomically.	200; one version increment; assignment and status history. P06, FR06.
TC09	Agent assigns employee/inactive account or unassigns In Progress ticket.	422; prior assignment and version preserved. FR06, P06.
TC10	Agent attempts Open to Resolved.	409 INVALID_TRANSITION; no event. P05.
TC11	Resolve In Progress without valid summary, then with valid summary.	First 422; second 200 Resolved with timestamp and history. FR07.
TC12	Reopen Resolved with valid reason.	Open; current resolution cleared; earlier resolution preserved. P07–P08.
TC13	Two agents patch using the same version.	One succeeds; other 412; no lost update. API02.
TC14	Patch without If-Match; submit unchanged values with current version.	First 428; second 200 with no version change or notification. P10, API02.

10 Acceptance tests continued

Test	Given and action	Expected result and trace
TC15	Employee comments on own Resolved ticket.	201; comment and event persisted; status stays Resolved; version increments. FR08.
TC16	Submit empty or 2001-character comment; then 1 and 2000 characters.	Invalid values fail with 422; boundary-valid values pass. FR08, Section 6.
TC17	Replay a create or comment with identical key/body; reuse key with different body.	Identical replay returns original result; changed body returns 409; no duplicates. API03.
TC18	Force database failure during notification insertion.	Entire mutation rolls back; retry can complete once. P09, NF07.
TC19	Change assignee and status together; check all affected accounts.	One notification per applicable recipient; no actor notification. N01.
TC20	User reads another user’s notification or marks it read.	404; original read_at unchanged. P04.
TC21	Mark own notification read twice.	Both 200; same read_at; ticket version unchanged. N02, P10.
TC22	Enter HTML/script-like content in title, description or comment.	Stored as text within limits; rendered inert; no script execution. NF01.
TC23	Lose network response after server commits creation; retry with same key.	One ticket; original reference recovered; form not cleared on initial failure. UI02, API03.
TC24	Use keyboard and narrow viewport for employee and agent journeys.	Labels, focus, errors and all actions usable. UI06, NF04.
TC25	Run defined 15-minute load and recover a backup in test.	Meet p95/error targets; restore within targets with recorded evidence. NF03, NF05.
TC26	Create/update from one session; observe another active authorized session.	History/status and notifications visible within refresh target. UI07, N01.
TC27	Try edits on Resolved ticket without reopening; submit resolution field out of context.	409 for forbidden resolved-ticket field changes; 422 for out-of-context reason fields. P07–P08.

Overall release acceptance

All S requirements and adopted P requirements shall be traced to passing tests. The employee creation-to-resolution journey and agent handling journey shall pass end to end. No unresolved defect may permit unauthorized access, lost committed data, duplicate writes within the retry window, or an invalid lifecycle transition. Record test environment, build identifier, execution date and evidence. This document specifies tests; it does not claim that any implementation has passed them.

11 Delivery plan and decisions

Implementation sequence

Task 1 — Establish identity integration, role enforcement and database schema. Deliver migrations and isolated test users. Verify access boundaries before exposing ticket data.

Task 2 — Implement creation, idempotency and scoped reads. Build New Ticket, My Tickets and detail screens. Verify required fields, Open status, persistence and ownership.

Task 3 — Implement agent assignment, priority and status updates with version checking. Add public comments and immutable history. Verify lifecycle transitions and concurrent updates.

Task 4 — Implement transactional in-app notifications and UI refresh. Verify recipient selection, deduplication and read ownership.

Task 5 — Complete error states, accessibility and operational controls. Execute all acceptance tests and document results. Resolve stakeholder decisions before production rollout.

Decisions requiring stakeholder review

Decision	Proposed baseline	Effect if changed
Identity and provisioning	Organizational identity provider; roles provisioned externally.	Changes authentication integration and deployment setup.
Update channels	In-app notifications only.	Email or other channels need delivery, retry and privacy contracts.
Workflow ownership	Agents control status; employees comment.	Employee reopen/close actions require new permissions and tests.
Resolution lifecycle	Three states; agents reopen; no Closed state.	Additional states require transition and retention changes.
Priority and assignment	Three priorities; manual assignment.	Escalations, auto-routing or SLAs expand scope.
Production operations	Proposed load and recovery targets in Section 9.	Owner must confirm sizing, service hours, retention and recovery duties.
Reporting and attachments	Excluded despite illustrative Reports menu.	Define separate requirements before implementation.

Specification change control

Store this specification with the implementation. For a requirement change, update the requirement, affected API/data contracts and tests before changing code. Record version, date, rationale and affected IDs. AI-generated code shall be reviewed against this baseline; a tool prompt does not override the specification.

Definition of ready

The brief is elaborated enough for an implementation exercise using the stated assumptions. For a production baseline, the product owner must resolve the decisions above and approve scope. The development team must name the technical stack and identity integration; the operational owner must accept hosting, retention and recovery responsibilities. No stakeholder approval is asserted by this draft.